

CSP Knowledge Base

What is CSP (Content Security Policy)?

Content Security Policy (CSP) is a **security feature in web browsers** that helps protect your web app from common attacks like:

- **XSS (Cross-Site Scripting)** – when malicious scripts are injected into your page.
- **Data injection attacks** – attackers adding untrusted content into your site.

Example of CSP

```
1 Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.kendostatic.com; style-src 'self'
  'unsafe-inline';
2
```

This means:

- `default-src 'self'` → By default, only load resources (images, fonts, etc.) from the same domain.
- `script-src 'self' https://cdn.kendostatic.com` → Only allow JavaScript from your own site and Kendo's CDN.
- `style-src 'self' 'unsafe-inline'` → Allow styles from your own site, but also inline styles (though `'unsafe-inline'` is not recommended unless necessary).

Inline Events and Content Security Policy (CSP)

When **Content Security Policy (CSP)** is enabled without allowing `'unsafe-inline'`, inline JavaScript (like `onclick="..."`) will not execute.

Example of Blocked Inline Event

```
1 <button onclick="alert('Hello!')">Click Me</button>
2
```

With CSP enforced, the above `onclick` handler will be ignored.

Recommended Approach: Use `addEventListener`

Instead of inline attributes, attach events in JavaScript code:

```
1 <button id="myBtn">Click Me</button>
2
3 <script>
4   const btn = document.getElementById("myBtn");
5   btn.addEventListener("click", () => {
6     alert("Hello!");
7   });
8 </script>
9
```

Common Inline Events and Their Safe Alternatives

Inline Event	<code>addEventListener</code> Equivalent
<code>onclick</code>	<code>element.addEventListener("click", fn)</code>
<code>ondblclick</code>	<code>element.addEventListener("dblclick", fn)</code>
<code>onchange</code>	<code>element.addEventListener("change", fn)</code>
<code>oninput</code>	<code>element.addEventListener("input", fn)</code>
<code>onfocus</code>	<code>element.addEventListener("focus", fn)</code>
<code>onblur</code>	<code>element.addEventListener("blur", fn)</code>
<code>onmouseover</code>	<code>element.addEventListener("mouseover", fn)</code>
<code>onmouseout</code>	<code>element.addEventListener("mouseout", fn)</code>
<code>onmouseenter</code>	<code>element.addEventListener("mouseenter", fn)</code>
<code>onmouseleave</code>	<code>element.addEventListener("mouseleave", fn)</code>
<code>onkeydown</code>	<code>element.addEventListener("keydown", fn)</code>
<code>onkeyup</code>	<code>element.addEventListener("keyup", fn)</code>
<code>onkeypress</code>	<code>element.addEventListener("keypress", fn)</code>
<code>onsubmit</code>	<code>element.addEventListener("submit", fn)</code>

<code>onreset</code>	<code>element.addEventListener("reset", fn)</code>
<code>onload</code>	<code>window.addEventListener("load", fn)</code>
<code>onunload</code>	<code>window.addEventListener("unload", fn)</code>
<code>onerror</code>	<code>window.addEventListener("error", fn)</code>

Kendo UI with Content Security Policy (CSP) in Core

When enabling **Content Security Policy (CSP)** in .NET Core, you must adjust your usage of Kendo UI. CSP restricts inline JavaScript, inline styles, and other unsafe practices.

1. Externalize JavaScript

- Move all logic into dedicated `.js` files.
- Avoid inline `<script>` blocks and `on*` event attributes (`onclick` , `onchange` , etc.).
- Regularly remove unused code for performance and maintainability.

✓ Example:

```

1 <!-- Instead of -->
2 <button onclick="deleteRow(1)">Delete</button>
3
4 <!-- Do this -->
5 <button id="delete-btn">Delete</button>
6
1 // In site.js
2 document.getElementById("delete-btn")
3   .addEventListener("click", () => deleteRow(1));
4

```

2. Avoid Inline Kendo Templates

Kendo's classic syntax (`#= ... #` , `# if ... #`) injects inline JavaScript, which CSP blocks.

✗ Not CSP compliant:

```

1 .ClientTemplate(
2     "#= (City == null) ? ' ' : City #" +
3     "<input type='hidden' name='Addresses[#=
4     courtEpoVictim.index(data)#].City' value='#= City #' />"
5 )

```

✓ CSP compliant (string interpolation with `Html.Kendo().Template()`):

```

1 .ClientTemplate(
2     Html.Kendo().Template()

```

```

3         .AddHtml("${data.City == null ? ' ' : data.City} " +
4             "<input type='hidden' " +
5             "name='Addresses[${courtEpoVictim.index(data)}].City'
6         " +
7             "value='${data.City ?? ' '}' />")
8     )

```

3. Use `.Deferred()` for Widgets

Always call `.Deferred()` in Razor views.

- Defers widget initialization until the bottom of the page.
- Keeps markup clean and **CSP-compliant**.
- Improves **performance** by reducing render-blocking scripts.

✓ Example:

```

1  @(Html.Kendo().Grid<MyModel>()
2      .Name("grid")
3      .Columns(columns =>
4          {
5              columns.Bound(x => x.City);
6          })
7      .Pageable()
8      .Sortable()
9      .Deferred() // <-- Important
10 )
11

```

4. Prefer JavaScript Functions Over Templates

Instead of embedding business logic in Kendo templates, define reusable functions in `.js` files.

✓ Example:

```

1  // helpers.js
2  function renderCity(city) {
3      return city ?? "N/A";
4  }
5
1  .ClientTemplate(Html.Kendo().Template()
2      .AddHtml("${renderCity(data.City)}"))
3

```

5. Avoid Kendo MVVM in Templates

Kendo does **not support observables inside templates**.

Using `kendo.observable` within templates often causes **unexpected behavior** in grids, dropdowns, etc.

✗ Don't:

```

1  <script type="text/x-kendo-template">
2      <span data-bind="text: Name"></span>
3  </script>
4

```

✓ Instead, bind via Grid columns or external JS functions.

6. Move Grids to Partial Views

For complex widgets, move markup into a **partial view** and keep initialization scripts in `.js` files.

- Better separation of concerns.
- Easier to maintain and reuse.
- Keeps Razor views clean.

✓ Example:

```
1 // _GridPartial.cshtml
2 @(Html.Kendo().Grid<MyModel>()
3     .Name("myGrid")
4     .Columns(columns =>
5     {
6         columns.Bound(x => x.City);
7     })
8     .Deferred()
9 )
10

1 // Parent View
2 @await Html.PartialAsync("_GridPartial")
3
```

7. CSP Headers Setup

Use [Joonasw.AspNetCore.SecurityHeaders](#) to configure CSP headers.

✓ Example:

```
1 app.UseCsp(csp =>
2 {
3     csp.ByDefaultAllow
4         .FromSelf();
5
6     csp.AllowScripts
7         .FromSelf()
8         .From("kendo.cdn.telerik.com")
9         .From("maps.googleapis.com")
10        .AddNonce()
11        .WithStrictDynamic();
12
13    csp.AllowStyles
14        .FromSelf()
15        .From("kendo.cdn.telerik.com")
16        .From("fonts.googleapis.com")
17        .AllowUnsafeInline();
18
19    csp.AllowFonts
20        .FromSelf()
21        .From("kendo.cdn.telerik.com")
22        .From("fonts.googleapis.com")
23        .From("fonts.gstatic.com data:");
24
25    csp.AllowImages
26        .FromSelf()
27        .From("kendo.cdn.telerik.com data:")
28        .From("maps.gstatic.com data:")
29        .From("maps.googleapis.com data:");
30
31    csp.AllowConnections
32        .To("maps.googleapis.com")
33        .To("kendo.cdn.telerik.com")
34        .To(Configuration["ContentSecurity:connect-src"])
35        .ToSelf();
36}
```

```

37     csp.OnSendingHeader = context =>
38     {
39         context.ShouldNotSend =
40         context.HttpContext.Request.Path.StartsWithSegments("/don't-apply");
41         return Task.CompletedTask;
42     };
43 };
44

```

8. Nonces for Inline Scripts and Styles

If you must allow inline snippets, use **nonces** instead of `unsafe-inline`.

Configure services:

```

1 services.AddCsp(nonceByteAmount: 32);
2

```

Apply nonce in Razor:

```

1 <script asp-add-nonce="true">
2     console.log("Hello, CSP-safe inline script!");
3 </script>
4

```

```

1 <style asp-add-nonce="true">
2     h1 { color: red; }
3 </style>
4

```

`@Html.Kendo().DeferredScripts()`

- Equivalent to `@Html.Kendo().DeferredScripts(true)`.
- Renders all collected widget initialization scripts.
- Typically placed once in `_Layout.cshtml` before the closing `</body>` tag.
- Ensures widgets are initialized after the page content has loaded.

✓ Example:

```

1 // Layout.cshtml
2 <body>
3     @RenderBody()
4
5     @Html.Kendo().DeferredScripts() <!-- Outputs all deferred widget scripts -->
6 </body>
7

```

`@Html.Kendo().DeferredScripts(false)`

- Suppresses script rendering completely.
- Even if you use `.Deferred()` on widgets, **no initialization code will be emitted**.
- Use this when:
 - You plan to initialize widgets **manually** in external JavaScript files.
 - You want **full control** over when and how Kendo scripts run.
 - You're integrating with a CSP setup that disallows auto-generated inline scripts.

